

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

Code:

```
#include <stdio.h>

int main() {
    int m, n, i, j;

    // Input memory blocks
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m], tempBlock[m];

    printf("Enter sizes of %d memory blocks:\n", m);
    for(i = 0; i < m; i++)
        scanf("%d", &blockSize[i]);

    // Input processes
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter sizes of %d processes:\n", n);
    for(i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    int allocation[n];
```

```
for(i = 0; i < m; i++)
```

```
    tempBlock[i] = blockSize[i];
```

```
for(i = 0; i < n; i++) {
```

```
    allocation[i] = -1;
```

```
    for(j = 0; j < m; j++) {
```

```
        if(tempBlock[j] >= processSize[i]) {
```

```
            allocation[i] = j;
```

```
            tempBlock[j] -= processSize[i];
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\n--- First Fit ---\n");
```

```
printf("Process No.\tProcess Size\tBlock No.\n");
```

```
for(i = 0; i < n; i++) {
```

```
    if(allocation[i] != -1)
```

```
        printf("%d\t%d\t%d\n", i+1, processSize[i], allocation[i]+1);
```

```
    else
```

```
        printf("%d\t%d\t\tNot Allocated\n", i+1, processSize[i]);
```

```
}
```

```
for(i = 0; i < m; i++)
```

```
    tempBlock[i] = blockSize[i];
```

```
for(i = 0; i < n; i++) {
```

```
    int best = -1;
```

```
    for(j = 0; j < m; j++) {
```

```
        if(tempBlock[j] >= processSize[i]) {
```

```
            if(best == -1 || tempBlock[j] < tempBlock[best])
```

```
                best = j;
```

```
        }
```

```

    }
    if(best != -1) {
        allocation[i] = best;
        tempBlock[best] -= processSize[i];
    } else {
        allocation[i] = -1;
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");
for(i = 0; i < n; i++) {
    if(allocation[i] != -1)
        printf("%d\t%d\t%d\t\t%d\n", i+1, processSize[i], allocation[i]+1);
    else
        printf("%d\t%d\t%d\t\tNot Allocated\n", i+1, processSize[i]);
}

for(i = 0; i < m; i++)
    tempBlock[i] = blockSize[i];

for(i = 0; i < n; i++) {
    int worst = -1;
    for(j = 0; j < m; j++) {
        if(tempBlock[j] >= processSize[i]) {
            if(worst == -1 || tempBlock[j] > tempBlock[worst])
                worst = j;
        }
    }
    if(worst != -1) {
        allocation[i] = worst;
        tempBlock[worst] -= processSize[i];
    } else {
        allocation[i] = -1;
    }
}

```

```

    }

}

printf("\n--- Worst Fit ---\n");

printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < n; i++) {

    if(allocation[i] != -1)

        printf("%d\t\t%d\t\t%d\n", i+1, processSize[i], allocation[i]+1);

    else

        printf("%d\t\t%d\t\tNot Allocated\n", i+1, processSize[i]);

}

return 0;

}

```

Output:

```

C:\Users\Admin\Desktop > .\program.exe
Enter sizes of 5 memory blocks:
400 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.   Process Size   Block No.
1             212             1
2             417             2
3             112             1
4             426             5

--- Best Fit ---
Process No.   Process Size   Block No.
1             212             4
2             417             2
3             112             3
4             426             5

--- Worst Fit ---
Process No.   Process Size   Block No.
1             212             5
2             417             2
3             112             1
4             426             Not Allocated

Process returned 0 (0x0)   execution time : 32.554 s
Press any key to continue.

```